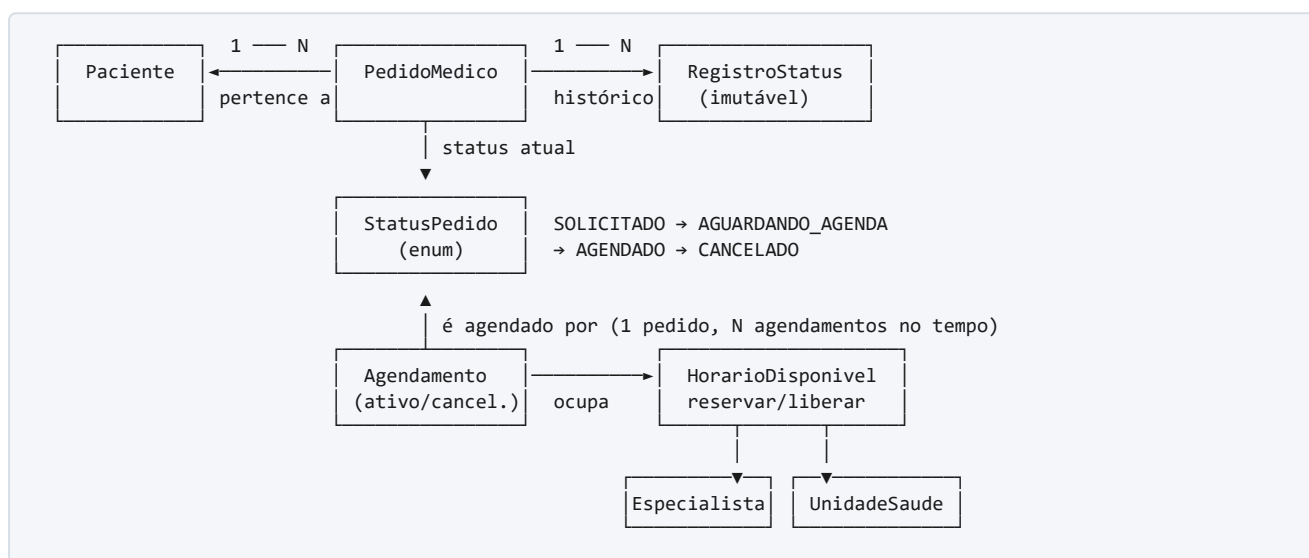


Parte 2 – Reflexão técnica sobre a modelagem

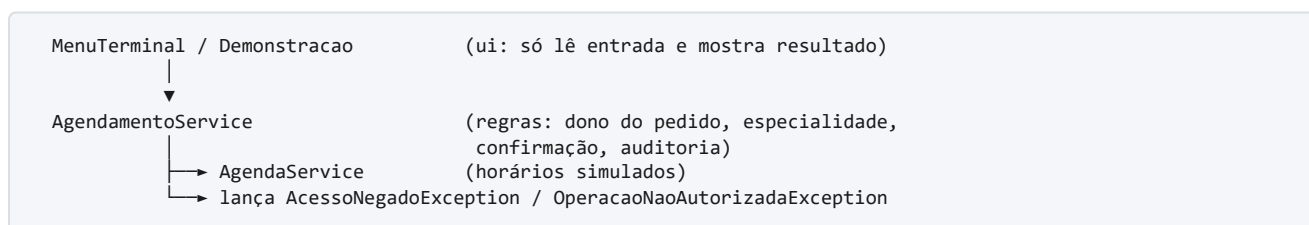
Este texto explica as decisões de modelagem do código Java da Sprint 1 (agendamento de pedidos médicos), o que elas resolvem e onde ainda são frágeis. As referências apontam para as classes em `src/br/com/hackgov`.

1. Visão geral do modelo

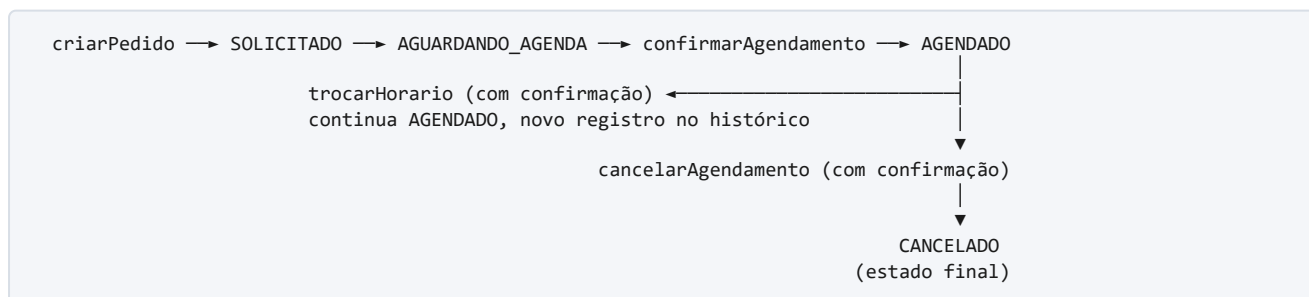
Como as classes se ligam:



Quem usa essas classes:



Caminho de um pedido, do começo ao fim:



O código está dividido em quatro camadas:

- **model/**: as entidades e as regras que dizem respeito a um único objeto (um horário sabe se está livre; um pedido sabe a quem pertence e registra o próprio histórico).

- **service/**: as regras que envolvem vários objetos (verificar o dono do pedido, conferir a especialidade do horário, exigir confirmação) e o log de auditoria.
- **exception/**: exceções próprias para as violações de segurança.
- **ui/**: o menu de terminal, que só chama o serviço e não contém regra de negócio.

Com essa divisão, a mesma regra vale para o menu interativo (`MenuTerminal`) e para os cenários automáticos (`Demonstracao`), porque os dois passam pelo `AgendamentoService`.

2. Decisões principais

2.1 Pedido e agendamento são entidades separadas

`PedidoMedico` representa a **necessidade** do paciente (preciso de um cardiologista). `Agendamento` representa a **alocação** dessa necessidade em um horário concreto. Poderíamos ter colocado o horário direto no pedido, mas separar permite que o pedido exista antes de haver vaga (status `AGUARDANDO_AGENDA`) e que um agendamento cancelado continue registrado (`ativo = false`) em vez de ser apagado. Em um serviço público isso é importante: o registro do que aconteceu não pode sumir.

2.2 O status é um `enum` que carrega a orientação ao paciente

`StatusPedido` guarda, junto de cada estado, uma descrição e uma mensagem de orientação (T07). Assim, a mensagem que o paciente lê fica sempre de acordo com o estado real do pedido, sem uma cadeia de `if/else` espalhada pela interface. Para criar um novo estado, basta alterar um único arquivo.

2.3 Um único ponto de mudança de status, com histórico imutável

O atributo `status` de `PedidoMedico` não tem setter. A única forma de alterá-lo é `alterarStatus(novo, responsavel, motivo)`, que sempre gera um `RegistroStatus`. O registro tem todos os campos `final` e o histórico é exposto como lista somente leitura (`Collections.unmodifiableList`). Com isso, a rastreabilidade (quem mudou, quando, de qual estado para qual e por quê) vem do próprio modelo e não depende de o programador lembrar de gravar um log.

2.4 A ocupação do horário fica dentro do próprio horário

`HorarioDisponivel` controla a sua disponibilidade com `reservar()` e `liberar()`, e `reservar()` lança exceção se o horário já estiver ocupado. Em `Agendamento.trocarHorario`, o novo horário é reservado **antes** de o antigo ser liberado. Se a reserva falhar, o paciente continua com o horário original e nunca fica sem agendamento.

2.5 Proteção do CPF no próprio modelo

`Paciente` não tem `getCpf()`. O CPF é normalizado (só dígitos) e validado no construtor. Fora da classe, ele só aparece mascarado (`getCpfMascarado`) ou é usado em comparações (`possuiCpf`). O log de auditoria identifica o paciente pelo `id` interno, nunca pelo CPF. Ou seja, a restrição foi colocada na classe, e não deixada como uma recomendação para quem escrever a interface.

2.6 Isolamento entre pacientes com um único ponto de verificação

Todas as operações que recebem um `pedidoId` (consultar status, confirmar, trocar, cancelar, ver histórico) passam por `buscarPedidoComAcesso`, que chama `PedidoMedico.pertenceA` e lança `AcessoNegadoException` quando o pedido é de outro paciente. A tentativa bloqueada também é registrada na auditoria. Como a verificação fica num único método privado, fica mais difícil criar uma operação nova e esquecer o controle de acesso. A listagem (`listarPedidos`) aplica o mesmo filtro.

2.7 Confirmação explícita e registro da origem da ação

`trocarHorario` e `cancelarAgendamento` recebem `confirmacaoPaciente` e `origem`. Sem confirmação, a operação é bloqueada, auditada e o agendamento original é mantido (cenário 4: o "Assistente IA" tenta trocar sozinho). Com confirmação, o histórico grava o responsável como "`<origem>` (confirmado pelo paciente)". Assim, fica registrado se a ação partiu do paciente ou de um assistente automatizado, o que responde à preocupação de não deixar uma IA alterar a agenda de alguém sem que a pessoa aprove.

2.8 Exceções separadas por tipo de erro

As violações de **segurança** usam exceções próprias (`AcessoNegadoException`, `OperacaoNaoAutorizadaException`). Os erros de **regra de negócio** ou de dado inválido usam as exceções padrão do Java (`IllegalStateException`, `IllegalArgumentException`). Com essa separação, uma camada futura pode, por exemplo, devolver HTTP 403 para as primeiras e 400/409 para as segundas, ou alertar a equipe só nos casos de segurança.

3. Limitações conhecidas

Algumas simplificações foram feitas de propósito, para o escopo da sprint, e precisam ser revistas antes de um uso real:

1. **As transições de status não são validadas por completo.** `alterarStatus` só impede mudanças a partir de `CANCELADO`. Nada impede, por exemplo, voltar de `AGENDADO` para `SOLICITADO`. O ideal é que cada estado declare quais estados podem vir depois dele (uma pequena máquina de estados dentro do `enum`).
2. **Cancelar o agendamento cancela o pedido inteiro.** Hoje, `cancelarAgendamento` leva o pedido para `CANCELADO`, que é um estado final. Se o paciente só quiser desmarcar e escolher outra data, ele precisa abrir um pedido novo. Uma alternativa seria voltar o pedido para `AGUARDANDO_AGENDA` e deixar `CANCELADO` apenas para a desistência do pedido.
3. **O estado SOLICITADO dura só um instante.** `criarPedido` passa o pedido direto para `AGUARDANDO_AGENDA`. O estado foi mantido porque é nele que entraria uma etapa futura de triagem ou regulação, e porque deixa no histórico o momento em que o pedido foi recebido.
4. **A especialidade é uma String.** A comparação ignora maiúsculas e minúsculas, mas um erro de digitação ("Cardiolgia") gera um pedido que nunca vai encontrar horário. O melhor seria um `enum` ou uma entidade `Especialidade` com código.
5. **A autenticação e a confirmação dependem de quem chama o serviço.** O serviço confia que o `Paciente solicitante` é realmente quem está usando o sistema e que o `boolean confirmacaoPaciente` veio do paciente. Em produção, os dois precisam vir de uma sessão autenticada e de um comprovante de confirmação (um código enviado ao paciente, por exemplo), e não de um parâmetro que qualquer código pode passar como `true`.

6. **Não há controle de concorrência.** `reservar()` verifica e altera a disponibilidade em dois passos, sem sincronização. Se dois pacientes confirmarem o mesmo horário ao mesmo tempo, os dois podem conseguir a vaga. Com banco de dados, isso seria resolvido com uma restrição de unicidade ou com bloqueio otimista.
7. **A auditoria guarda textos em memória.** O log é uma `List<String>`, sem estrutura (tipo de evento, ator, alvo) e sem proteção contra alteração. Ele serve para a demonstração, mas uma auditoria de verdade precisa ser persistida, consultável e somente de inserção.
8. **O tempo é lido diretamente do relógio do sistema.** O uso de `LocalDateTime.now()` dentro das entidades dificulta testar regras que dependem de data. Receber um `java.time.Clock` resolveria isso.
9. **Tudo fica em memória e não há testes automatizados.** Os IDs são contadores no serviço e os dados somem quando o programa fecha. A classe `Demonstracao` funciona como roteiro de cenários, mas não substitui testes JUnit que verifiquem os resultados automaticamente.

4. Próximos passos sugeridos

Prioridade	Ação	Resolve
Alta	Máquina de estados em <code>StatusPedido</code> (<code>podeIrPara</code>)	Limitação 1
Alta	Testes JUnit para os cenários 1 a 7	Limitação 9
Alta	Persistência (JPA/banco) com restrição de unicidade no horário reservado	Limitações 6 e 9
Média	Cancelar agendamento voltando o pedido para <code>AGUARDANDO_AGENDA</code>	Limitação 2
Média	<code>Especialidade</code> como <code>enum</code> /entidade	Limitação 4
Média	Auditoria estruturada e persistida	Limitação 7
Baixa	Injeção de <code>Clock</code>	Limitação 8

5. Conclusão

O modelo prioriza três características que são obrigatórias em um sistema público de saúde: **rastreabilidade** (histórico imutável e um único ponto de mudança de status), **privacidade** (CPF encapsulado e isolamento entre pacientes) e **controle do paciente sobre a própria agenda** (confirmação explícita e registro de quem iniciou cada ação). Essas regras ficam nas classes de modelo e no serviço, e não na interface, por isso continuam valendo quando o menu de terminal for trocado por uma API ou por um aplicativo. As limitações listadas acima são, na maior parte, consequência da ausência de persistência e de autenticação reais, e a modelagem atual já tem os pontos certos para receber essas melhorias.